# Full-Stack 4D Path Planning for U-Space: Architecture and Scalability Evaluation

Sukhbir Singh[1,2], Enno Nagel[2], Bianca Isabella Schuchardt[2]

[1]Technical University of Munich (TUM), Germany
[2]DLR Institute of Flight Guidance, Germany

**Abstract**

Urban Air Mobility (UAM) requires scalable path-planning solutions to safely integrate large numbers of eVTOL vehicles into structured low-altitude airspace. U-space, the European framework for unmanned aircraft traffic management in low-altitude airspace, relies on trajectory-based planning services to support such operations. Although routing algorithms, 4D trajectory generation, and conflict detection are well established individually, few studies report end-to-end performance results of an integrated implementation. This paper presents a full-stack 4D path-planning framework for U-space corridor operations that combines A*-based global routing on a corridor graph, CFMS-based time-parameterized trajectory generation, and NDMap-based conflict detection. The system follows a microservice-oriented architecture separating a topology service providing vertiports and corridor networks, a planner service performing routing and trajectory generation, and an interface to a U-space Service Provider (USSP). Computational performance is evaluated on a representative vertiport corridor network using sequential trajectory requests, with runtime explicitly separated into routing, trajectory generation, and total processing time.

## 1 Introduction

Urban Air Mobility (UAM) is emerging as a promising extension of future transportation systems. To enable the safe integration of large numbers of electric vertical take-off and landing (eVTOL) vehicles into low-altitude airspace, structured traffic-management frameworks such as the European U-Space [1, 9] and the U.S. UTM concept [14] have been proposed. These initiatives emphasize service-oriented architectures and trajectory-based operations to ensure scalable and safe airspace integration [2, 16].

A central requirement in such environments is scalable real-time path planning. In dense urban scenarios, large numbers of trajectory requests must be processed while ensuring separation minima and compliance with structured corridor constraints. Search-based approaches such as A* have been widely investigated for UAV routing [13, 15], alongside 4D grid-based conflict management strategies [3]. Conflict detection in multi-dimensional trajectory spaces has been studied in trajectory-based operations frameworks [8, 10], and surveys summarize the range of available UAV planning methods [4].

In operational U-Space systems, however, path planning is tightly coupled with trajectory generation [12], conflict detection [10], and verification services [16]. Monolithic implementations of these components may limit scalability and extensibility. In operational U-Space systems, however, path planning is tightly coupled with trajectory generation [12], conflict detection [10], and verification services [16]. Monolithic implementations of these components may limit scalability and extensibility. Microservice architectures offer a modular alternative, enabling loosely coupled services with independent scalability [7, 11]. In the architecture considered in this work, services communicate through RESTful HTTP interfaces and exchange structured data using JSON messages. This approach allows individual components—such as topology services, planning services,

and authorization interfaces—to be deployed and scaled independently while maintaining clear interface boundaries. Nevertheless, quantitative evaluations of integrated, microservice-based U-Space planning systems remain limited, particularly with respect to latency and scalability under traffic load.

This paper presents a microservice-oriented path-planning architecture and evaluates its baseline computational performance. The system integrates (i) a global A*-based planner, (ii) a 4D trajectory-generation pipeline incorporating vehicle and temporal constraints, and (iii) an embedded conflict-detection mechanism inspired by multi-dimensional trajectory management concepts [10, 12]. A CesiumJS-based frontend enables interactive visualization of time-dependent trajectories [6].

The contributions of this work are:

- Implementation of an integrated path planning stack combining A* routing, CFMS-based trajectory generation, and NDMap conflict detection.
- Runtime breakdown analysis separating routing, trajectory synthesis, and total processing time.
- Scalability evaluation on a Hamburg-based vertiport corridor network under sequential request scenarios.

The remainder of the paper is organized as follows. Section 2 reviews related work, Section 3 describes the system design, Section 4 outlines the evaluation setup, and Section 5 presents the results.

## 2 Related Work

### 2.1 U-Space and 4D Trajectory-Based Operations

The SESAR Master Plan situates U-Space within the broader transition toward trajectory-based operations (TBO) and digitally enabled air traffic management [2]. Comparable developments in FAA NextGen emphasize negotiated 4D trajectories to improve predictability and capacity [8]. These initiatives highlight time-constrained trajectory management and interoperable services as key enablers of scalable urban air traffic integration.

Conflict management in discretized 4D space–time representations has been demonstrated for multi-UAV operations [3]. Furthermore, architectural concepts for U-Space services propose modular stacks integrating monitoring and traffic management functions [5]. While these works provide important conceptual foundations, empirical performance evaluations of deployable end-to-end systems remain limited.

### 2.2 UAV Path Planning and Conflict Detection

Search-based planners, particularly A*-based methods, remain widely used for low-altitude UAV routing due to their deterministic behavior and manageable computational complexity [13, 15]. Studies comparing 2D and 3D search highlight trade-offs between runtime and path optimality, with higher-dimensional search increasing computational cost.

Conflict detection in multi-dimensional trajectory spaces has been extensively studied within trajectory-based operations frameworks [8, 10]. These approaches formalize separation assessment in time-dependent state spaces and provide a foundation for integrated 4D conflict management.

Although algorithmic aspects of routing and conflict detection are well established, many studies evaluate these components in isolation rather than as parts of distributed, service-oriented architectures operating under traffic load.

### 2.3 Microservice Architectures in UAV Systems

Microservice architectures have gained prominence in distributed software engineering due to their modularity and independent scalability [7]. Applications in UAV-related domains demonstrate benefits in system maintainability and flexible deployment [11].

However, the integration of search-based routing, 4D trajectory synthesis, and conflict detection within a unified microservice-based U-Space stack has received comparatively limited quantitative evaluation, particularly with respect to latency behaviour and scalability under increasing demand.

**Research Gap.** Existing literature establishes theoretical and architectural foundations for U-space integration and trajectory planning. However, few works report end-to-end latency measurements of a deployable path planning stack that integrates routing, trajectory generation, and conflict checking. In particular, runtime breakdowns separating graph-based routing from physically feasible trajectory synthesis are rarely quantified. This paper addresses that gap by evaluating the computational performance of a full-stack implementation under a realistic vertiport network scenario.

## 3 Methodology

### 3.1 System Architecture and Operational Flow

The proposed framework is implemented as a set of loosely coupled services that together generate and validate conflict-free 4D trajectories for structured U-Space corridor operations. The main logical components are: (i) a *Topology Registry* (Webserver) providing vertiports and the corridor network, (ii) a *Planner* responsible for route computation and iterative conflict checking, and (iii) a *U-space Service Provider (USSP)* responsible for flight-plan validation, authorization, and supervision. In the present implementation, NDMap-based conflict detection is executed within the Planner process rather than deployed as a separate service; it is depicted as a distinct logical block in Fig. 1 for clarity of responsibilities.

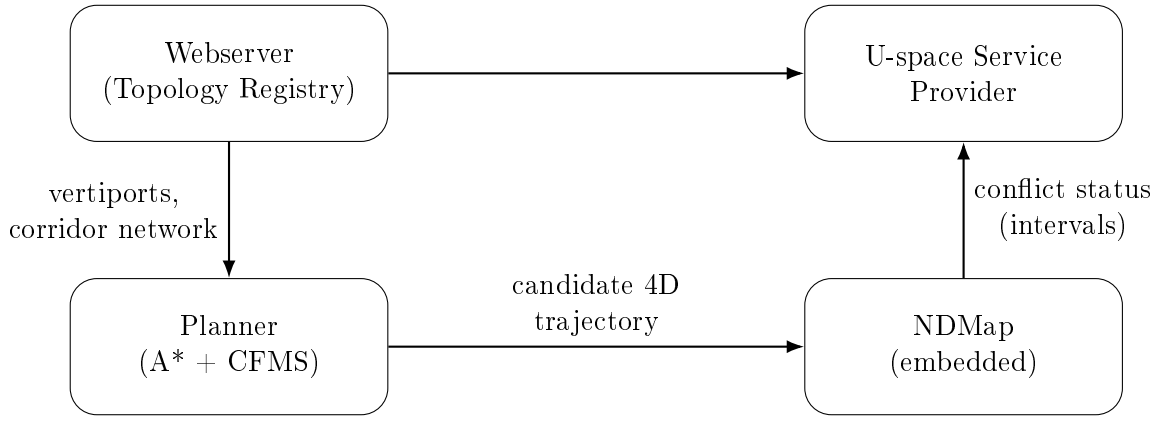



Figure 1: High-level architecture of the full-stack design. NDMap is logically embedded within the Planner workflow.

Fig. 1 illustrates the high-level service interactions. The detailed internal planning workflow, including iterative conflict detection and lateral re-routing, is depicted in Fig. 2.

### 3.2 Global Route Planning on Corridor Graphs

Global routing between vertiports is formulated as a shortest-path problem on a weighted graph $G = (V, E)$, where nodes $V$ represent vertiports and edges $E$ represent available corridor connections. Edge weights correspond to geodesic distance between the connected endpoints (Haversine

distance). A* search is used to compute the shortest route between the selected origin and destination. The heuristic is the straight-line geodesic distance to the destination, which is admissible and yields efficient convergence. The output of this stage is an ordered list of corridor nodes:

$$P = [v_0, v_1, \ldots, v_n],$$

which serves as the lateral (2D) reference route for subsequent trajectory generation.

### 3.3 4D Trajectory Generation via CFMS

The route $P$ is transformed into a flyable four-dimensional trajectory (latitude, longitude, altitude, time) using a waypoint synthesis step and a downstream trajectory generator. The implementation interfaces a Python-based Planner with the C++-based CFMS flight-management and 4D trajectory generation system [12].

**Inputs.** The trajectory generation stage consumes: (i) the node sequence $P$, (ii) vertiport coordinates and elevations, (iii) a fixed cruise altitude layer. In this study, the cruise layer is defined as a fixed absolute altitude of 200 m above mean sea level (MSL). This abstraction does not account for local terrain variation or dynamic weather effects; these factors are excluded from the current evaluation scope., and (iv) nominal phase-dependent speeds.

**Waypoint synthesis and phase annotation.** A compact waypoint list $W = [w_1, \ldots, w_m]$ is synthesized from $P$ by adding a small number of vertical-profile waypoints: a departure waypoint at the origin pad elevation, an intermediate climb waypoint, cruise waypoints aligned with corridor nodes at the cruise altitude layer, a descent-initiation waypoint, and a final waypoint at the destination pad elevation. Each waypoint is annotated with a nominal phase label (takeoff, cruise/transition, landing) and an associated target speed. Nominal speeds are derived from vehicle performance limits scaled by a safety factor to preserve flyability margins.

**CFMS execution and subsampling.** The annotated waypoint list is converted into the CFMS input format (WGS-84 coordinates, altitudes, phase and speed fields) and passed to CFMS to produce a dense, time-parameterized trajectory $\mathcal{T}$ satisfying the kinematic profile [12]. To reduce payload size for downstream processing and visualization, $\mathcal{T}$ is uniformly subsampled to a fixed maximum number of points while retaining the first and last samples.

### 3.4 Conflict Detection in 4D Using NDMap

Conflicts are detected using NDMap [10], configured over the 4D state space $(\lambda, \varphi, H, t)$ (longitude, latitude, altitude, time). NDMap indexes previously accepted trajectories and efficiently identifies segments of an inspected candidate trajectory that violate separation minima with respect to the active set.

**Configuration and output.** Separation requirements are parameterized by minimum spacing thresholds in each dimension (horizontal, vertical, and temporal). For each detected conflict, NDMap returns one or more time intervals $[t_{\text{start}}, t_{\text{end}}]$ indicating when the candidate violates separation with any active trajectory. These intervals are used as the primary input to the resolution loop.

### 3.5 Conflict Resolution by Lateral Re-routing

Conflict resolution is implemented directly within the planning loop using *lateral re-routing* on the corridor graph. The procedure is iterative: (1) compute a candidate route using A*, (2) generate a 4D trajectory and perform conflict detection using NDMap, (3) if conflicts are present, penalize

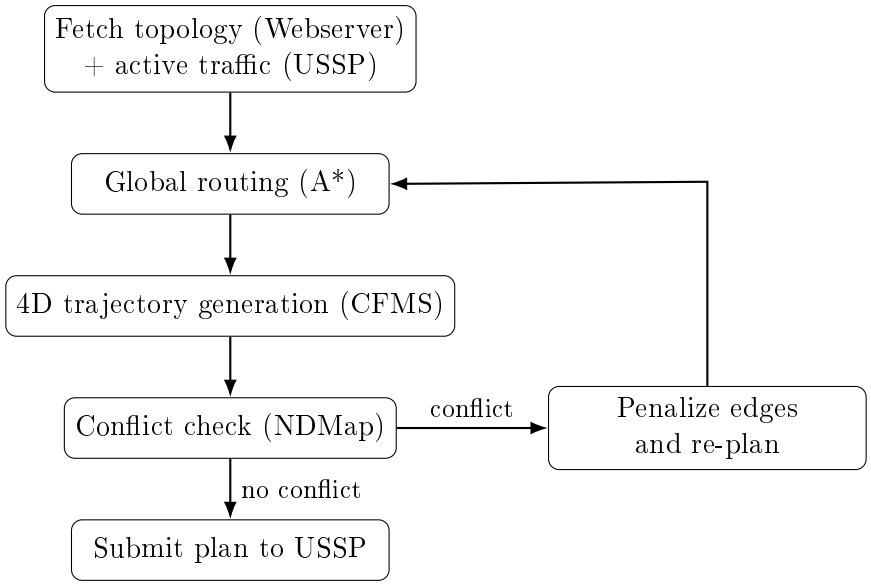



Figure 2: Operational planning loop. Conflicts trigger lateral re-routing and re-execution of A*.

corridor edges belonging to the conflicting candidate route by assigning them a very large cost (effectively removing them from consideration), and (4) re-run A* on the modified graph. The loop terminates when NDMap reports no conflicts or when no feasible alternative route exists.

This strategy is intentionally conservative and simple: it preserves the overall system modularity and enables scalable online admission control while illustrating how embedded 4D conflict checks can be integrated into a microservice-oriented planning pipeline. More expressive resolution strategies (e.g., temporal shifts or vertical maneuvers) are left for future work.

# 4 Experimental Setup

## 4.1 Scenario and Network Model

All experiments were conducted on the Hamburg vertiport network considered in this work. Vertiports are represented as graph nodes corresponding to averaged pad coordinates, and air corridors are modeled as undirected edges forming a connected graph. Each corridor operates at a fixed absolute cruise layer of 200 m above mean sea level (MSL). Terrain elevation and obstacle variation are not modeled in this study; the corridor layer is defined in an MSL reference to ensure reproducibility of the computational evaluation. Edge weights are defined by the geodesic (Haversine) distance between connected vertiports.

This abstraction enables reproducible shortest-path routing while preserving geometrical realism at the network level.

## 4.2 System Configuration

All experiments were executed on a workstation equipped with an Intel Core i7-8650U CPU @ 1.90 GHz (4 cores, 8 threads) and 16 GB RAM under a 64-bit operating system (x64 architecture).

The Planner (Python implementation), CFMS trajectory generator (C++), and NDMap conflict detection module were executed locally on the same machine. No distributed deployment, containerization, or artificial network latency was introduced. Consequently, reported runtimes represent baseline computational performance under single-machine execution. The system is architected according to microservice principles with clearly defined service boundaries; however, the experiments in this paper evaluate baseline performance with all components executed on a single host to isolate computational effects from network latency.

### 4.3 Planner and Generator Configuration

- **Global planner:** A* search on the undirected corridor graph, using Haversine distance both as edge cost and as admissible heuristic $f(n) = g(n) + h(n)$.
- **Cruise altitude:** Fixed absolute cruise layer of 200 m (MSL reference), independent of local pad elevation and terrain variation.
- **Trajectory generation:** CFMS-based 4D synthesis with phase-dependent nominal speeds derived from vehicle performance limits scaled by a safety factor.
- **Sampling:** Generated trajectories were uniformly subsampled to a fixed maximum number of points to reduce payload size.

### 4.4 Evaluation Methodology

To evaluate baseline computational performance, batches of trajectory requests were processed sequentially. For scalability analysis, 100 independent origin–destination vertiport pairs were randomly sampled from the corridor graph. Each trajectory request was fully completed before the next was initiated, ensuring that measured runtimes were not influenced by parallel processing effects.

In the baseline scalability experiment, conflict detection and resolution mechanisms were disabled to isolate the computational cost of global routing and trajectory generation. Runtime measurements were recorded separately for:

- A* path computation,
- 4D trajectory generation, and
- total end-to-end processing time per request.

These measurements provide insight into the relative computational contributions of routing and trajectory synthesis within the proposed framework.

## 5 Results

## 6 Discussion

## 7 Conclusion

## Acknowledgments

## References

[1] U-space blueprint. Technical report, European Commission, 2017. URL https://ec.europa.eu/research/participants/documents/downloadPublic?documentIds=080166e501ba2683&appId=PPGMS.

[2] Sesar master plan 2025. Technical report, SESAR Joint Undertaking, 2025. URL https://www.sesarju.eu/sites/default/files/documents/reports/SESAR%20Master%20Plan%202025.pdf.

[3] José Joaquín Acevedo, Carlos Capitán, Jesirs Capitiin, Angel R. Castaño, and Aníbal Ollero. A geometrical approach based on 4d grids for conflict management of multiple uavs operating in u-space. In *International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 263–270, 2020. doi: 10.1109/ICUAS48674.2020.9213929.

[4] Ali Haidar Ahmad, Oussama Zahwe, Abbass Nasser, and Benoit Clement. Path planning algorithms for unmanned aerial vehicle: Classification, performance, and implementation. In *3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, pages 1–6, 2023. doi: 10.1109/ICECCME57830.2023.10252168.

[5] Carlos Capitán, Héctor León, Jesús Capitán, Ángel Rodríguez Castaño, and Anibal Ollero. Unmanned aerial traffic management system architecture for u-space in-flight services. *Applied Sciences*, 11:3995, 2021. doi: 10.3390/app11093995.

[6] CesiumGS. Cesiumjs: 3d geospatial visualization for the web, 2025. URL `https://cesium.com/platform/cesiumjs/`.

[7] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. *Microservices: Yesterday, Today, and Tomorrow*, pages 195–216. Springer International Publishing, 2017. doi: 10.1007/978-3-319-67425-4_12. URL `https://doi.org/10.1007/978-3-319-67425-4_12`.

[8] Gabriele Enea and Marco Porretta. A comparison of 4d-trajectory operations envisioned for nextgen and sesar, some preliminary findings. 5:4152–4165, 2012.

[9] European Union Aviation Safety Agency (EASA). Easy access rules for u-space — may 2024. Technical report, 2024. URL `https://www.easa.europa.eu/en/downloads/139563/en`.

[10] Alexander Kuenz. *High Performance Conflict Detection and Resolution for Multi-Dimensional Objects*. Number 31 in DLR Forschungsbericht. October 2015. URL `https://elib.dlr.de/98476/`.

[11] Lea Matlekovic, Filip Juric, and Peter Schneider-Kamp. Microservices for autonomous uav inspection with uav simulation as a service. *Simulation Modelling Practice and Theory*, 119: 102548, 2022. doi: 10.1016/j.simpat.2022.102548. URL `https://www.sciencedirect.com/science/article/pii/S1569190X22000466`.

[12] Fabian Morscheck. A modular experimental flight management and 4d trajectory generation system for unmanned multicopter, urban air mobility vehicles and other vtol vehicles. In *IEEE/AIAA Digital Avionics Systems Conference (DASC)*, pages 1–9, 2021. doi: 10.1109/DASC52595.2021.9594290.

[13] Carlos Augusto Pötter Neto, Gustavo de Carvalho Bertoli, and Osamu Saotome. 2d and 3d a* algorithm comparison for uas traffic management systems. In *International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 72–76, 2020. doi: 10.1109/ICUAS48674.2020.9214028.

[14] Thomas Prevot, Joseph Rios, Parimal Kopardekar, John Robinson III, Marcus Johnson, and Jaewoo Jung. Uas traffic management (utm) concept of operations to safely enable low altitude flight operations. In *AIAA Aviation Forum*, 2016. doi: 10.2514/6.2016-3292.

[15] Laith R. Sahawneh, Matthew E. Argyle, and Randal W. Beard. 3d path planning for small uas operating in low-altitude airspace. In *International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 413–419, 2016. doi: 10.1109/ICUAS.2016.7502528.

[16] SESAR Joint Undertaking. Sesar concept of operations (conops 2019). Technical report, 2019. URL `https://ec.europa.eu/research/participants/documents/downloadPublic?documentIds=080166e5c91e877d&appId=PPGMS`.